

Freight Rate Prediction: Approach and Validation

1. Summary

The task is to predict the posted rate for 12,000 loads in November and December 2025 from 48,000 labelled loads dated January to October. The validation set is entirely in the future, so I validated by time rather than at random.

The final model is a gradient-boosted tree on static load attributes (distance, weight, equipment, pickup and delivery coordinates), trained on rate per mile after setting aside 1.4% of labels that look corrupt, followed by a one-parameter linear correction for how far that day's market_index sits from its month's average. On four forward-in-time folds (July to October) it reaches a median absolute percentage error of 1.75% and a mean absolute error of \$101 per load, against 3.20% and \$135 for a per-mile lookup baseline.

The most useful finding was about market_index: used naively it made out-of-time predictions worse, even though it looked informative. Section 5 explains what is actually going on.

2. Data findings and how they were handled

Issue	What I found	What I did
Negative weights	292 train and 145 validation rows. weight runs 5,000 to 47,500 and the negatives mirror that range.	Treated as sign errors: used the absolute value.
Missing values	weight: 300 train, 165 validation. market_index: 374 train, 249 validation (about 0.8%).	weight filled with the training median. market_index recovered from the other loads on the same date.
Weight ceiling	2.5% of loads sit at exactly 47,500 in both files.	Left as is. It looks like a cap and the model sees it equally in both periods.
Corrupt-looking rates	682 train rows (1.42%) are 4+ robust sigma from a robust fit: 341 about 2 to 5 times too high, 341 about 0.2 to 0.3 times too low. Spread evenly over months and equipment.	Excluded from fitting inside each fold. Kept in every scoring metric, since the evaluator's labels will contain them too.
Odd distances	111 rows (0.23%) have a recorded distance far above the straight-line distance from the coordinates.	Tested as a feature (road/straight-line ratio); it changed nothing, so I dropped it.
Unseen cities	8 validation cities never appear in training (Allentown, Charlotte, Chicago, Jackson, Knoxville, Laredo, Norfolk, San Diego). 12.1% of loads touch one.	No city identity in the model. Geography enters only through coordinates, which tested better on held-out cities.
Unused columns	quote_signal has 0.05 correlation with rate per mile and is mostly row-to-row noise.	Tested twice and left out (section 5).

The December chart file contains no market_index or quote_signal. That constraint shaped the design: a model that needs those columns could not produce the required chart without inventing them.

3. Train/validation split

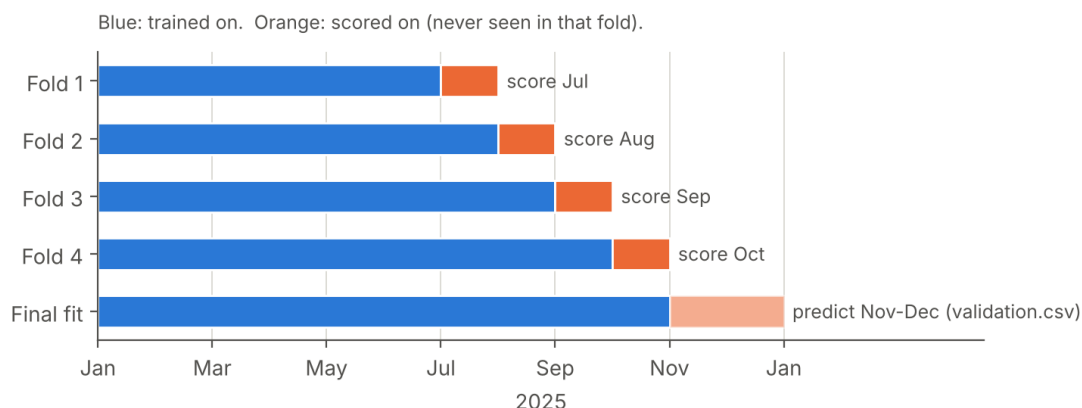


Figure 1. Expanding-window validation. Each fold trains on every month up to k and scores month $k+1$. Folds 1 to 4 score July to October.

The validation file starts on 1 November, after the last training date, so the real task is forecasting the next two months. A random split would let the model train on days that surround each test day and would reward memorising a particular day. Section 5 shows this is not hypothetical here: the same feature is judged in opposite ways by a random split and by a forward-in-time split.

Features are computed without labels, and the date-level `market_index` average only looks at rows with the same date. Corrupt-label trimming is fitted on each fold's training rows only. `tests/test_sanity.py` checks that test dates are strictly later than training dates and that the features are identical with the label column removed.

4. Model

- **Target:** log of rate per mile. Rate scales with distance, so predicting the per-mile rate removes most of the variance and keeps errors proportional. Predictions are converted back to dollars by multiplying by distance.
- **Stage 1:** scikit-learn `HistGradientBoostingRegressor` on log distance, `|weight|`, equipment and the four coordinates. It never sees the date, so it cannot memorise individual days.
- **Stage 2:** a Huber-loss linear regression of stage 1's out-of-fold residuals on the day's `market_index` minus its month average. Using out-of-fold residuals keeps the correction honest; in-sample residuals are too small.
- **Why trees at all:** the lookup baseline and a robust linear model both lose clearly (table 1). Regional effects in the coordinates need a nonlinear model. Hyperparameters barely matter: all 12 settings I tried land between 2.10% and 2.11% clean MAPE.

5. Experiments

Scoring convention. All numbers are the mean of the four forward-in-time folds. "All rows" includes the corrupt-looking labels, which no model can predict. "Clean rows" excludes them and is how I compare models, because the corrupt rows add the same unavoidable noise to every candidate. The clean-row flag comes from a robust fit to all training data, so it is a diagnostic for reporting, not something used to train.

5.1 Does the model beat sensible baselines? (table 1)

Model	MAE \$	Median APE	Clean MAE \$	Clean MAPE
Median rate per mile by equipment and distance band	134.8	3.20%	81.2	3.71%

Model	MAE \$	Median APE	Clean MAE \$	Clean MAPE
Robust linear (distance, weight, equipment)	118.9	2.58%	65.1	2.94%
GBM, raw labels	108.2	2.03%	54.3	2.30%
GBM, absolute-error loss, raw labels	105.2	1.90%	51.3	2.15%
GBM, corrupt labels set aside	104.2	1.86%	50.3	2.11%
Final: GBM + month-deviation market adjustment	101.0	1.75%	47.0	1.99%

Table 1. Temporal cross-validation, mean of four folds. Setting aside corrupt labels beat both raw training and an absolute-error loss that was meant to be robust.

5.2 How should market_index be used? (table 2)

The static model's out-of-fold daily residual correlates 0.70 with that day's market_index (304 days), so the signal is real. But handing it to the model as an ordinary input made predictions worse.

Variant	MAE \$	Median APE	Clean MAE \$	Clean MAPE
No market input	104.2	1.86%	50.3	2.11%
Joint GBM + absolute market_index	129.1	3.04%	75.5	3.17%
Joint GBM + deviation from month average	107.7	1.98%	53.8	2.27%
Two-stage, absolute market_index	126.4	2.98%	72.7	3.11%
Two-stage, deviation from month average (final)	101.0	1.75%	47.0	1.99%
Two-stage, deviation from 31-day centred average	100.1	1.69%	46.1	1.95%
Two-stage, deviation + day of week	102.8	1.79%	48.7	2.07%
Two-stage, deviation + quote_signal	101.5	1.82%	47.6	2.05%

Table 2. Ways of using market_index and quote_signal.

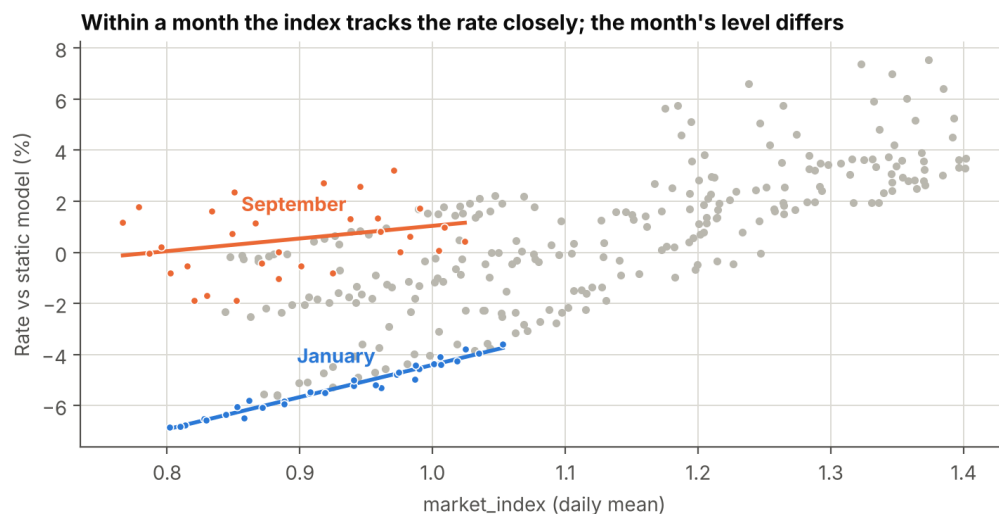


Figure 2. Out-of-fold daily rate residual against market_index. In six of the ten months the two move together with correlation above 0.95, but each month sits at its own level. In January and September the slopes are similar and the levels differ by about 6 points.

That is why the absolute index fails: it implies a level that does not carry over to a new month. The day's deviation from its own month's average is the part that does transfer, and a one-parameter correction on it

improves the four-fold average. Adding day of week or quote_signal did not help. A 31-day centred reference was slightly better (1.95% vs 1.99% clean MAPE); I kept the calendar-month version because it is simpler to explain and the difference is small.

The split itself also matters. With a random 5-fold split, adding the absolute market_index looks like a large improvement (clean MAPE 2.92% to 2.13%); with the forward-in-time split it is a large loss (2.11% to 3.17%). A random split would have led me to ship the wrong model.

5.3 Other decisions

- **Static inputs:** removing weight costs 0.85 points of clean MAPE, removing coordinates 0.74, removing equipment 2.9. All three stay.
- **City identity:** no gain on seen cities (2.11% vs 2.12%) and worse on held-out cities (3.43% vs 2.97%), so the model uses coordinates only.
- **Forecast scaling:** multiplying predictions by 0.98 to 1.05 never beat 1.00 by more than \$0.3 MAE, so I left the forecast uncalibrated.
- **Unseen cities cost little:** holding out whole cities gives 2.97% clean MAPE, against 2.92% for a random split over the same months, so coordinates generalise to new cities about as well as to known ones. The two numbers are not comparable with the forward-in-time folds, which cover the easier July to October months.

6. Fixed December prediction

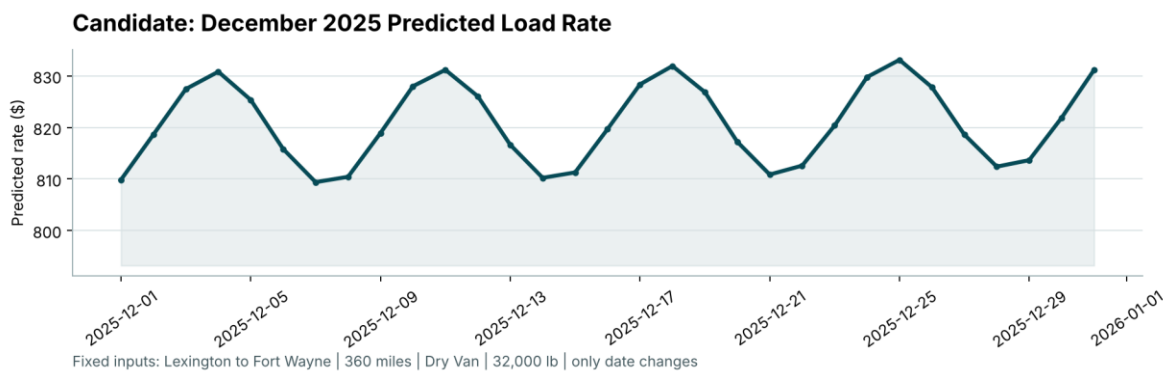


Figure 3. Chart produced by the provided score.py from the final model (Lexington to Fort Wayne, 360 miles, Dry Van, 32,000 lb).

Predictions range from \$809 to \$833, mean \$821. The lane's historical Dry Van median was roughly \$780 to \$880 by month (21 loads), so the level is consistent with history.

How to read the shape. Only the date varies, so every wiggle comes from stage 2. The chart file has no market_index, so I used each date's value from validation.csv, which is a feature column and contains no labels. The weekly cycle (Thursdays highest, Sundays lowest) reflects the day-of-week pattern in that index. The curve does not model Christmas or New Year: the training data has no holiday evidence, and the 25 December peak is just a Thursday.

Assumption to flag. If the intent was a pure forecast with no knowledge of December's index, the honest curve is nearly flat at about \$821. The adjustment moves the predictions by at most about 1.5% around that level.

7. Error analysis

On the October fold (trained January to September), median error is 1.48% over all rows and 1.44% on clean rows. Mean error is 4.7% overall because 80 corrupt-looking labels average 182% error each. Observations:

- Short hauls under 300 miles are the weakest segment: 7.0% mean error versus 2 to 5% in the other distance bands. A fixed dollar error is a larger share of a small rate.
- Reefer loads have the highest mean error (5.4% vs 4.4% for Dry Van and Flatbed), but the median errors are similar across equipment types, so this is mostly the corrupt labels.
- Only 8 October loads were on a lane unseen in training, too few to say anything about unseen lanes from this fold. The city-holdout experiment (5.3) covers that question instead.

8. Limitations and what I would do next

- **Monthly level is unpredictable from this data.** Month-average residuals have a standard deviation of about 3 points (from -5% in January to +5% in June) and `market_index` does not explain that. With one year of data I cannot tell whether November and December carry a seasonal offset; a prior year would settle it. This is the biggest risk to the validation score.
- **Corrupt labels.** About 1.4% of validation labels will be unpredictable noise. If the unseen metric is RMSE or MAE they dominate the total, so differences between good models may look small.
- **Model selection used the same four folds that are reported.** The decisions that matter are large (2.11% vs 3.17%), but the small differences, such as 1.95% vs 1.99%, should not be over-read.
- **Next steps:** a prior year to estimate seasonality; an explanation for the corrupt labels from the data owner; prediction intervals; monitoring the stage-2 slope for drift.

9. Reproduction

`python -m pip install -r requirements.txt`, place the provided CSVs in `data/`, then run `src/data_checks.py`, `src/experiments.py` (about 10 minutes), `src/train_predict.py` and `score.py` as described in `README.md`. Everything is seeded and deterministic.